

SVG Implementation Guide For Bag Packing Configurator

Purpose

This document describes only the SVG concepts required for the Bag Packing Configurator project.

The goal is to render:

- Bag image
- Pocket images
- Selection outlines
- Rotation
- Dragging
- Zooming
- Panning

No advanced SVG drawing features are required.

SVG Coordinate System

SVG uses a 2D coordinate system.

Origin:

```
``text id="ffx8hz" (0,0)
```

```
is located at:
```

```
``text id="u20lwr"  
Top Left
```

Example:

```
``text id="uxob26" x increases →
```

y increases ↓

Root SVG

Application contains a single SVG viewport.

Example:

```
```tsx id="0b4u7x"
<svg
 width="100%"
 height="100%"
 viewBox="0 0 1000 800"
>
</svg>
```

---

## Why ViewBox

ViewBox allows:

- Zoom
- Pan
- Responsive sizing

without changing object coordinates.

Example:

```
```tsx id="l1r1n7" viewBox="0 0 1000 800"
```

represents the virtual coordinate system.

Layer Structure

Render order matters.

Recommended:

```
```text id="h7x0d7"
SVG
```

```
└─ Background
|
└─ Bag Image
|
└─ Pocket Layer
|
└─ Selection Layer
|
└─ Interaction Layer
```

---

## Bag Image

Bag is rendered using SVG image element.

Example:

```
``tsx id="Op7f5e" <image href={bag.imageUrl} x={0} y={0} width={bagWidth} height={bagHeight} />
```

```
Bag image is visual only.
```

```
Packing calculations do not use image dimensions.
```

```

```

```
Pocket Rendering
```

```
Pockets are rendered using SVG image elements.
```

```
Example:
```

```
``tsx id="xwn6hb"
<image
 href={pocket.imageUrl}
 x={100}
 y={50}
 width={300}
 height={200}
/>
```

Image dimensions are calculated from physical dimensions.

---

# Important Rule

Packing engine uses:

```
```text id="d6j0yj" widthCm heightCm
```

Rendering uses:

```
```text id="mr7n98"  
widthPx
heightPx
```

converted through scale factor.

---

## Pocket Group

Every pocket should be wrapped inside a group.

Example:

```
```tsx id="0iqgic" <g>
```

```
<image />
```

```
</g>
```

Reason:

Future transforms are applied to the group.

Transform System

Transforms should be applied on group level.

Example:

```
```tsx id="s4k8ek"  
<g transform="translate(100 50)">
```

```
<image />
</g>
```

This moves the pocket.

---

## Rotation

Rotation should occur on group.

Example:

```
```tsx id="h7kry6" <g transform=" translate(100 50) rotate(90) "
```

```
<image /> </g>
```

Supported rotations:

```
```text id="d4kq8x"
```

0°

90°

for POC.

---

## Recommended Pocket Component

Conceptually:

```
```tsx id="6j6kt6" <g>
```

```
<image />
```

```
<rect />
```

```
</g>
```

Where:

image = visual product

```
rect = selection outline

---

# Selection Outline

When pocket is selected:

Render rectangle outline.

Example:

```tsx id="t4x7qq"
<rect
 x={0}
 y={0}
 width={width}
 height={height}
 fill="none"
 stroke="blue"
/>
```

Used only for:

- Selection
- Hover
- Validation

Not for actual rendering.

---

## Collision Debug Layer

Optional developer mode.

Example:

```
```tsx id="h9khk3" <rect fill="none" stroke="red" />
```

```
Displays packing footprint.

Useful for debugging.
```

```
---
```

```
# Dragging

Dragging updates:

```ts id="8p4jv9"
x
y
```

stored in state.

SVG simply re-renders.

Example:

```
```tsx id="9f7v7j" translate(x y)
```

```
updates automatically.
```

```
---
```

```
# Zoom System
```

```
Zoom should be implemented using:
```

```
```text id="e65zje"
viewBox
```

not CSS transform.

Example:

```
```tsx id="jcdjwr" viewBox=" panX panY zoomWidth zoomHeight "
```

```
Benefits:
```

- Crisp rendering
- Better performance
- Consistent coordinates

```
---
```

```
# Pan System
```

```
Pan updates:
```

```
``ts id="2xpvbw"  
viewBox.x  
viewBox.y
```

Example:

```
``tsx id="1ptmp6" viewBox=" 100 50 1000 800 "
```

```
---  
  
# Hit Testing  
  
Mouse selection should use:  
  
``text id="sj4vzg"  
onPointerDown  
onPointerMove  
onPointerUp
```

attached to pocket groups.

Example:

```
``tsx id="34oh3i" <g onPointerDown={...}
```

```
---  
  
# Coordinate Conversion  
  
Screen coordinates must be converted to SVG coordinates.  
  
Utility:  
  
``ts id="qtm4ah"  
clientToSvg()
```

Required for:

- Dragging
- Selection
- Placement

Image Aspect Ratio

Must remain locked.

Example:

```
```tsx id="7gimq8" <image preserveAspectRatio="none" />
```

Do NOT use.

---

Use:

```
```tsx id="81gg4z"
<image />
```

with calculated width and height.

Dimensions come from:

```
```text id="9n4wkl" Physical Size
```

not image pixels.

---

# Rendering Order

Recommended:

```
```text id="2ky4f2"
Bag
```

↓

Placed Pockets

↓

Selected Pocket

↓

Drag Preview

↓

Debug Layer

Selected pocket should always render on top.

Future Features

SVG architecture should support:

- Multiple bag templates
- Multiple pocket types
- Reserved zones
- Export PNG
- Export PDF
- Touch interaction
- Multi-compartment bags

without changing rendering fundamentals.

SVG Elements Used In This Project

Only these SVG elements are required:

```
``text id="8hffiy" svg g image rect
```

Everything else is unnecessary for POC.

No need for:

```
``text id="gzlj49"  
path  
polygon  
mask  
clipPath  
filter  
foreignObject  
text
```

unless future requirements demand them.

Final SVG Architecture

```
```text id="bh7jpw" svg
```

```
|— Bag Image | |— Pocket Layer | |— Pocket Group | | |— Product Image | | |— Selection Rect
| | | |— Pocket Group | |— Drag Preview Layer | |— Debug Layer ```
```

This structure should be considered the standard rendering architecture for the Bag Packing Configurator.